CookMem Tutorial 4

Debugging Without Valgrind

Cheap memory checks you can turn on when you need them



For services and applications that Valgrind cannot run, or would slow down too much

When Valgrind doesn't fit

20-30x

slower under Valgrind: timing changes, timeouts break

Windows

is not supported by Valgrind

Special build

and much more memory for AddressSanitizer

CookMem's checks are part of the allocator

- Normal builds
- Per context
- On when you need them

What CookMem catches, and what it doesn't

Catches

Writes just past the end
with padding, when the memory is freed

Double frees
unless the memory was reused in between

Leaks
forEachAllocation () lists what was not freed

Growth
StatsLogger counts bytes, peaks and segments

Does not catch

Out-of-bounds reads
reads are never checked

Use after free
reading or writing freed memory

Writes before the start
padding is only after the end

Foreign pointers
freeing memory from somewhere else

Padding catches writes past the end

One allocation in memory




```
cookmem::SimpleMemContext<> memCtx (cookmem::MAX_PADDING_SIZE);

char* name = static_cast<char*> (memCtx.allocate (5));
std::memcpy (name, "hello", sizeof ("hello"));   // 6 bytes

memCtx.deallocate (name);   // throws MEM_ERROR_PADDING
```

examples/01_padding_check.cpp (shortened)

1. **1 to 8 bytes**
   Set when the context is created.

2. **Exact sizes**
   getSize () returns the requested size.

Handling the exception

```
try
{
    memCtx.deallocate (name);
}
catch (const cookmem::Exception& ex)
{
    // MEM_ERROR_PADDING: padding byte got modified.
    log (ex.getError (), ex.getMessage (),
         ex.getFile (), ex.getLine ());

    // Do not use memCtx again: stop or restart this
    // request, which owns the context.
}
```

examples/01_padding_check.cpp (shortened)

1. **getError ()**
   MEM_ERROR_PADDING, MEM_ERROR_DOUBLE_FREE, ...

2. **Do not continue**
   The context must not be used again.

3. **Contain it**
   Restart only the request that owned it.

Double frees

```
void* buffer = memCtx.allocate (100);
memCtx.deallocate (buffer);

memCtx.deallocate (buffer);   // throws MEM_ERROR_DOUBLE_FREE
```

examples/02_double_free.cpp (shortened)

Always on

In every build, with or without padding.

When it can, and cannot, be detected

Detected:

`free (p)` `free (p)` → **throws**

Not detected:

`free (p)` `allocate ()` `free (p)` → **frees the new allocation**

A leak report for each request

```
Context memCtx (cookmem::MAX_PADDING_SIZE);
for (int id = 1; id <= 6; ++id)
{
    handleRequest (memCtx, id);

    memCtx.forEachAllocation ([id] (void*, std::size_t size)
    {
        std::cout << "Request " << id << " leaked "
                  << size << " bytes\n";
    });
    memCtx.releaseAll ();   // the next request starts clean
}
```

examples/03_leak_report.cpp (shortened)

```
Request 3 leaked 203 bytes
Request 6 leaked 206 bytes
2 of 6 requests leaked
```

1. **Per request**
   A leak is tied to the request that made it.

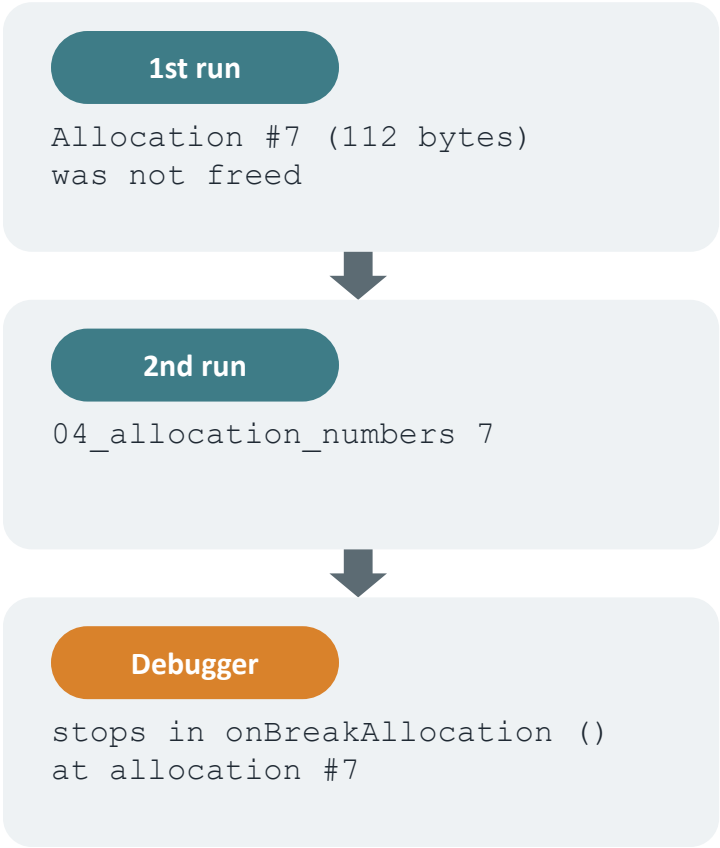
2. **Exact sizes**
   Padding shows the requested sizes.

Finding where a leak was allocated

```
void logAllocation (void* ptr, std::size_t,
                    cookmem::MemAllocType_et)
{
    unsigned long number = ++m_next;
    m_numbers[ptr] = number;
    if (number == m_breakOn)
        onBreakAllocation (number);   // set a breakpoint here
}
```

examples/numbering_logger.h (shortened)

A logger is a template parameter. It implements the same functions as cookmem::StatsLogger, so the calls are inlined.



Statistics show memory that grows

```
typedef cookmem::SimpleMemContext<cookmem::MmapArena,
                                  cookmem::StatsLogger> Context;
Context memCtx;

const cookmem::MemStatistics& stats =
    memCtx.getLogger ().getStats ();
stats.getCurrentAllocated ();
stats.getPeakAllocated ();
```

examples/05_statistics.cpp (shortened)

Byte counts: the sizes getSize () reports, rounded up to the alignment unless exact sizes are stored.

Bytes in use

Phase 1	10,000
Phase 2	20,000
Phase 3	30,000

Turn the checks on at startup

```
MemoryConfig config = MemoryConfig::fromStartup (argc, argv);

cookmem::MmapArena arena;
NumberingLogger logger (config.debug);
Context memCtx (arena, logger,
                config.debug ? cookmem::MAX_PADDING_SIZE : 0);

// ... handle requests ...
if (config.debug)
    memCtx.forEachAllocation (reportLeak);
```

examples/06_debug_config.cpp (shortened)

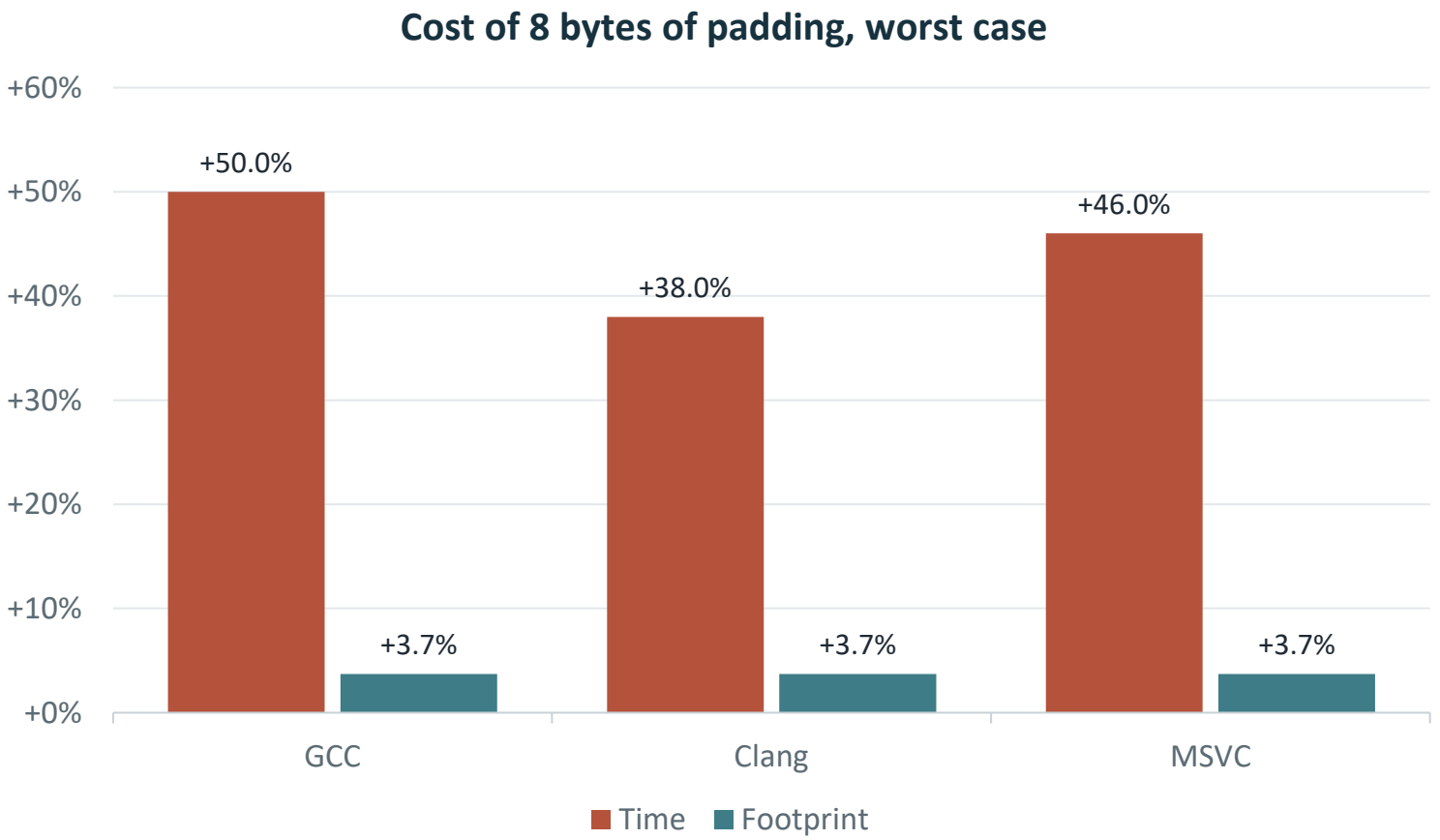
Normal run

```
$ 06_debug_config
Normal run: no checks
```

Debug run

```
$ 06_debug_config --debug-memory
Request 3: allocation #6
(200 bytes) was not freed
```

What padding costs



+38% to +50% time

per allocate and free pair, in a program that does nothing else.

+3.7% memory

for 10,000 allocations of 16 to 271 bytes.

In a real program, allocation is a small share of the time, so the total cost is much smaller.

examples/07_padding_cost.cpp on an AMD Ryzen 7 5800H; GCC 15.2 and Clang 21.1 on Ubuntu 26.04 (WSL2), MSVC 19.39 on Windows 11; best of three runs, Release builds.

Know the limits

!

Checked at free time

A write past the end is reported when the memory is freed, and not at all if it is released all at once.

!

One error ends the context

Log it, then stop or restart the work that owned the context.

!

A complement, not a replacement

Use Valgrind or AddressSanitizer when you can; they catch reads and use after free.

Summary

1. **Padding and double-free checks**
   Catch writes past the end and double frees, in normal builds.

2. **Leak reports with numbers**
   forEachAllocation () and a numbering logger find what leaked, and where.

3. **Turn them on at startup**
   Normal runs pay almost nothing; debug runs reproduce the problem.



Next: Tutorial 5, Limiting Memory Use · github.com/coconut2015/cookmem-tutorials